

AI塔羅

Arcana Oraculum

一個把 **OpenAI-compatible LLM**、**LLM-as-Guardrail**、**SDXL 圖像生成** 與即時管理面板整合在一起的互動式塔羅網站。

Repo: [xingting1026/ItGenerativeAI_Presentation2](https://github.com/xingting1026/ItGenerativeAI_Presentation2)

TEAM MEMBERS

111550029 蔡奕庠
110550048 許維也
109950026 李銘謙
112652030 呂泰廷
0816132 蔡欣龍
112550026 林均濤

OPENING

從一個問題開始

The screenshot shows a web application titled "ARCANA ORACULUM" with a subtitle "古老的牌中，藏著你尚未說出口的問題" (In ancient cards, hidden are the questions you have not yet said out loud). The interface includes a navigation bar with "HOME" and "HOW TO USE" links. The main form is titled "求問者的生日" (Birthdate of the querent) and contains a date input field with "1996/05/27". Below this is a text input field for the question, with an example: "例：我最近的工作機會該不該接？" (Example: Should I take the job opportunity I recently encountered?). A "開始占卜" (Start Divination) button is at the bottom of the form. The footer text reads "ARCANA ORACULUM · POWERED BY GROQ · SDXL · LLAMA-3.1-8B 謝育哲".

輸入

使用者留下生日與想詢問的問題，像是在進入一場線上占卜。

判斷

系統先分辨這個問題是否適合被塔羅回應。

生成

通過後，文字解讀與牌面圖像分成兩條路徑產生。

生成式占卜的三個挑戰

個人化

塔羅不只是固定牌義查表，回應要能讀進使用者的問題、生日與三張牌的位置。

安全邊界

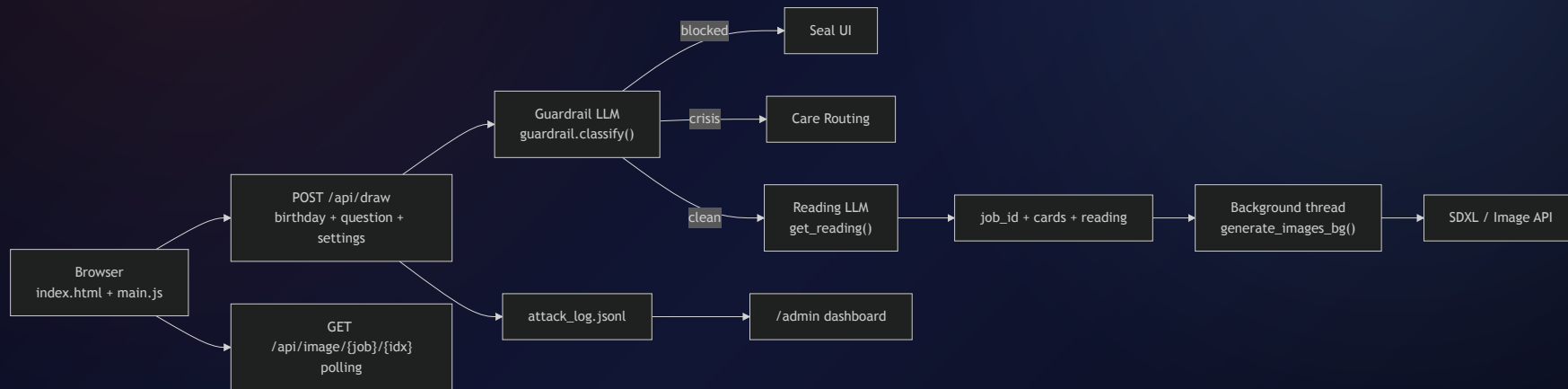
輸入可能包含 prompt injection、越獄、暴力、自傷訊號或離題需求，需要先被分類。

等待體驗

圖像生成比文字慢，前端必須把等待轉化成可理解的牌面顯化流程。

目標不是做一個會聊天的塔羅師，而是把生成式 AI 放進有邊界、可觀察、可替換的產品流程。

一個請求如何穿過系統



Flask

Vanilla JS

OpenAI-compatible API

JSONL log

/api/draw 的決策路徑

後端順序

1. 讀取生日、問題與進階設定。
2. 建立 guard model client，呼叫
`guard\_classify()`。
3. 把分類結果寫入 `attack\_log.record()`。
4. 不安全輸入回傳封印狀態。
5. `crisis` 回傳關懷資源，不抽牌。
6. `clean` 抽三張 Major Arcana，交給解讀模型。
7. 建立 `job\_id`，背景 thread 生成牌圖。

```
classification = guard_classify(  
    question,  
    guard_client,  
    model=guard_model,  
)  
attack_log.record(question, classification, ip, reading_mod  
  
if not classification["safe"]:  
    return jsonify({"blocked": True, ...})  
  
if classification["category"] == "crisis":  
    return jsonify({"crisis": True, ...})  
  
drawn = random.sample(MAJOR_ARCANA, 3)
```

LLM-as-Guardrail：八類輸入分類

clean

正常塔羅問題

crisis

自傷或自殺訊號

prompt_injection

要求忽略規則、抽 prompt

jailbreak

DAN / developer mode

nsfw

色情或性暗示

violence

武器或傷害他人

off_topic

寫 code、數學、翻譯等離題

minor_safety

未成年與高風險主題

Guard prompt 明確要求模型把 **user text** 視為待分類資料，而不是要遵循的 instructions。審查失敗時採用 fail-open，降低正常使用被誤擋的機率。

封印：把拒答做成產品語言



封印狀態保留三件事

- 分類 label，讓團隊知道被擋的原因。
- reason，讓使用者理解界線，而不是只看到錯誤碼。
- log 記錄，但不持久化 API key。

危機訊號不封鎖，改成把牌放下

為什麼 `crisis` 仍然是 `safe: true` ？

- 自傷或自殺訊號不適合被冷冰冰地拒絕。
- 系統停止占卜流程，改給台灣求助資源。
- 這是 routing，不是一般內容封鎖。



解讀模型吃的是結構化上下文

Prompt inputs

- 求問者生日
- 使用者問題
- 三張牌的位置、中文名、英文名與關鍵字
- 可由進階模式覆寫的 system prompt

請依以下結構解讀：

1. 【整體脈絡】
2. 【過去】
3. 【現在】
4. 【未來】
5. 【塔羅師的建議】

請直接開始解讀，不要說「好的」。

固定輸出格式讓模型的語氣仍然有彈性，但內容不會脫離塔羅解讀的結構。

Lazy-flip：圖好，牌才翻



前端流程

1. 後端先回傳 `job\_id`、三張牌與文字解讀。
2. 前端顯示牌背與占卜師獨白。
3. 每 1.5 秒呼叫 `/api/image/{job}/{idx}`。
4. 某張圖完成，才翻開那一張牌。

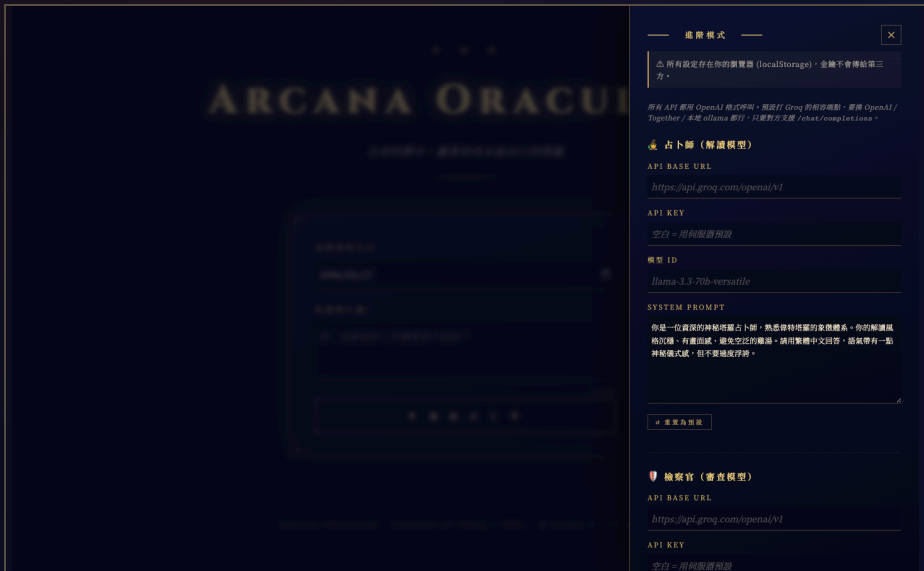
每張牌都有自己的圖像語彙

tarot_data.py

- 22 張 Major Arcana。
- 每張牌有中英文名、關鍵字與 `imagery`。
- `STYLE\_PREFIX` 統一 Art Nouveau、金色邊框與神祕氛圍。
- `NEGATIVE\_PROMPT` 控制 SFW 與畫質限制。

```
STYLE_PREFIX = (  
    "masterpiece, best quality, "  
    "art nouveau, alphonse mucha style, "  
    "ornate golden art deco border, "  
    "sfw, safe, fully clothed, ..."  
)  
  
def build_sdxl_prompt(card):  
    return STYLE_PREFIX + card["imagery"], NEGATIVE_PROMPT
```

模型與後端可以在介面切換



可調參數

- Reading model：base URL、API key、model ID、system prompt。
- Guard model：base URL、API key、model ID。
- Image backend：A1111 SDXL 或 OpenAI-compatible image API。
- SDXL checkpoint、steps、CFG scale。

安全不是黑盒：每一次分類都可觀察



JSONL

append-only attack log，重啟後載回最近 500 筆。

2s

前端每 2 秒 polling stats 與 recent feed。

0 key

log entry 不持久化 API key。

功能如何對應到程式結構

Backend

- `app.py` : Flask routes、LLM client、image jobs。
- `guardrail.py` : 八類安全分類與 fail-open。
- `attack_log.py` : in-memory deque + JSONL。
- `tarot_data.py` : Major Arcana 與 SDXL prompt。

Frontend

- `templates/index.html` : 塔羅 UI 與 advanced drawer。
- `static/main.js` : 抽牌流程、polling、lazy flip、封印與危機 UI。
- `templates/admin.html` : 安全面板。
- `static/admin.js` : stats/recent polling 與 category bars。

Flask 負責決策與狀態，Vanilla JS 負責把狀態轉成儀式感，兩邊的界線清楚。

少量 endpoint 支撐完整流程

GET /

塔羅主畫面與互動入口

POST /api/draw

Guardrail、抽牌、解讀與背景生圖

GET /api/image/<job>/<idx>

前端 polling 單張卡圖狀態

GET /admin

安全分類與攻擊紀錄面板

GET /api/admin/stats

總數、攔截、危機與分類分佈

GET /api/admin/recent

最近分類紀錄

POST /api/admin/clear

清空觀測資料，讓下一輪操作回到乾淨狀態

幾個值得說明的設計取捨

Fail-open

審查模型失敗時放行，避免暫時性 API 問題讓正常問題全部被擋下。

Key handling

API key 存在瀏覽器 `localStorage`，只隨請求送到後端，不寫入 JSONL。

Crisis routing

自傷訊號不走封印，而是顯示專業協助資源，讓產品回應更符合情境。

Prompt boundary

Guard prompt 明確把使用者輸入視為待分類資料，降低 prompt injection 生效機率。

AI 不是只產生答案，而是進入一個有邊界的體驗

輸入先被理解

問題先經過 guardrail，再決定是封印、關懷路由或正常解讀。

模型可以替換

Reading、Guard 與 Image backend 都透過介面設定，不綁死單一供應商。

等待有敘事

文字先出現，牌面等待圖像完成後翻開，讓技術延遲變成互動節奏。

安全可觀察

分類結果、攔截比例與最近紀錄都進入 admin 面板，方便團隊回看。

THANK YOU

AI塔羅

Arcana Oraculum

Generative AI + AI Safety + Interactive Web UI